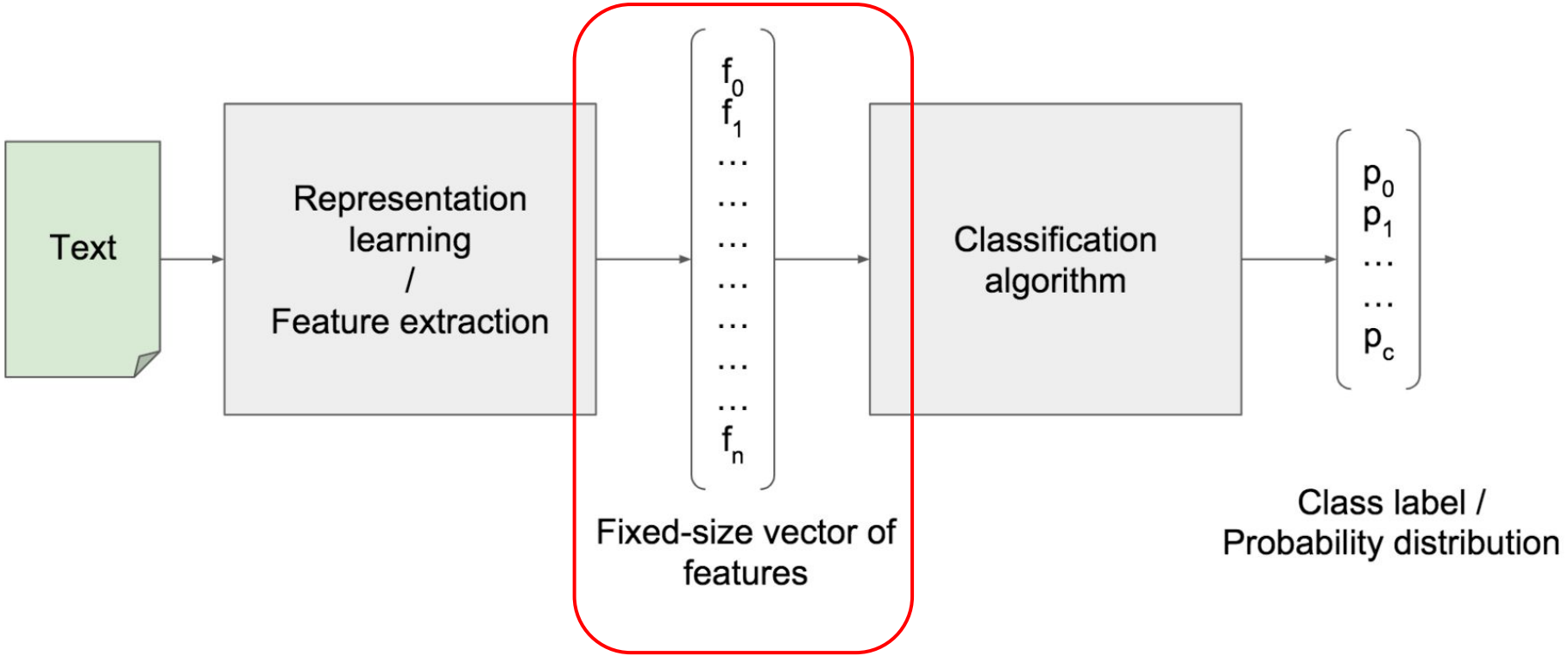
Reccurrent Neural Networks. Seq2seq Models.

Nikolay Karpachev
4.03.2026

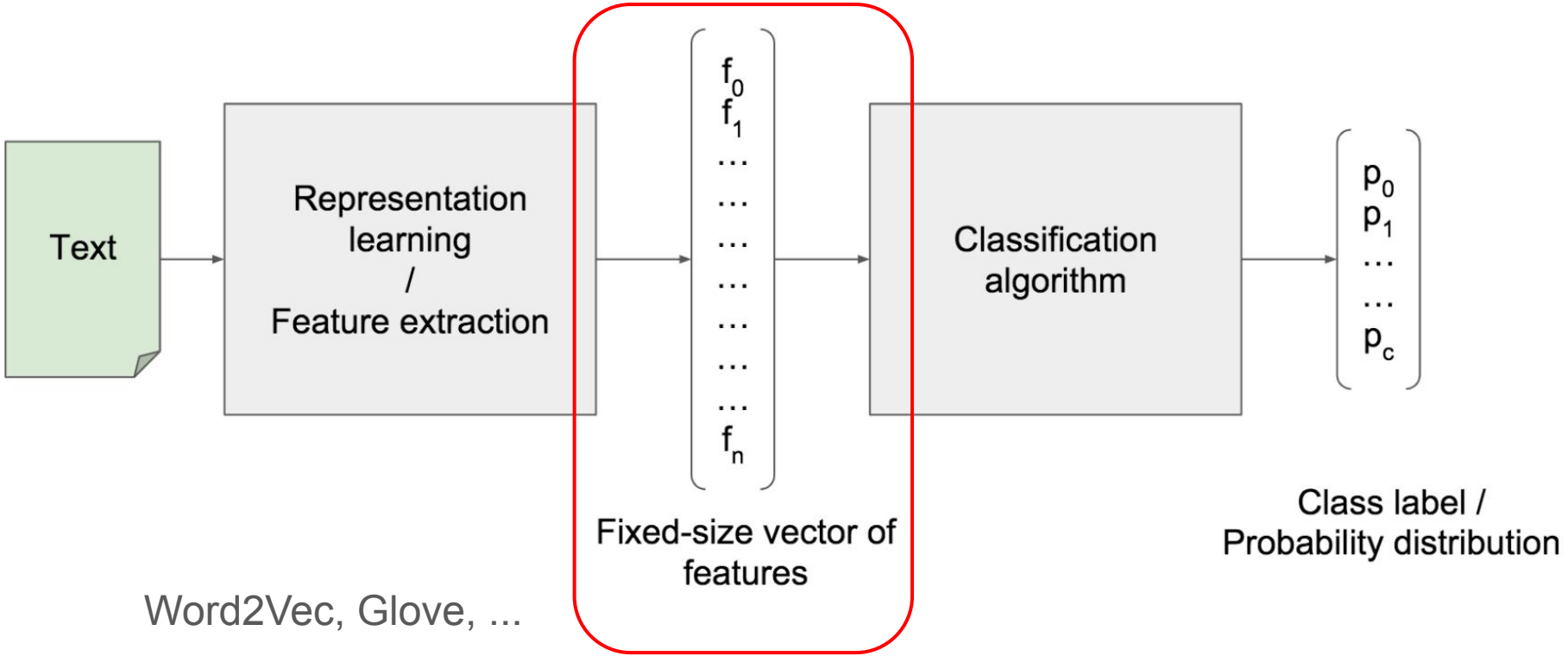
Outline

- From features to algorithms
  - Ways of working with textual data
  - RNN
  - “RNN+”, LSTM
- Generative NLP
  - Seq2Seq
  - Decoding

Text classification in general

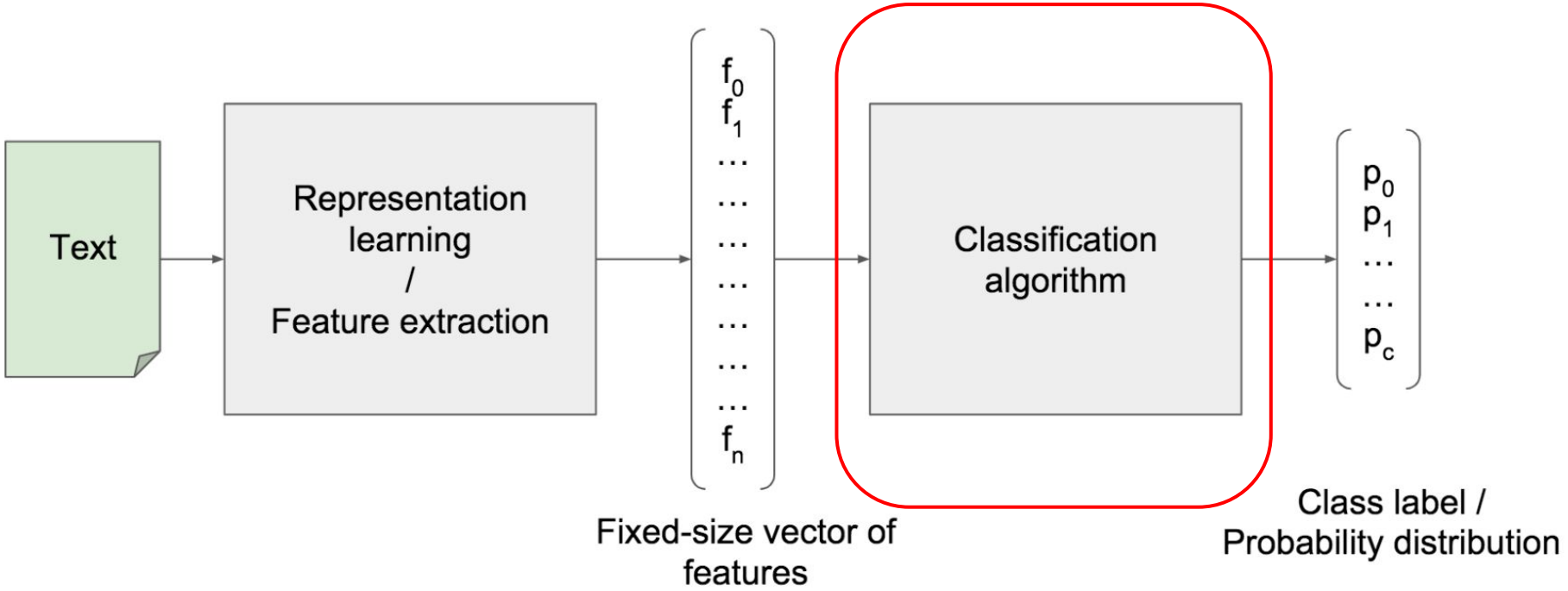


Text classification in general



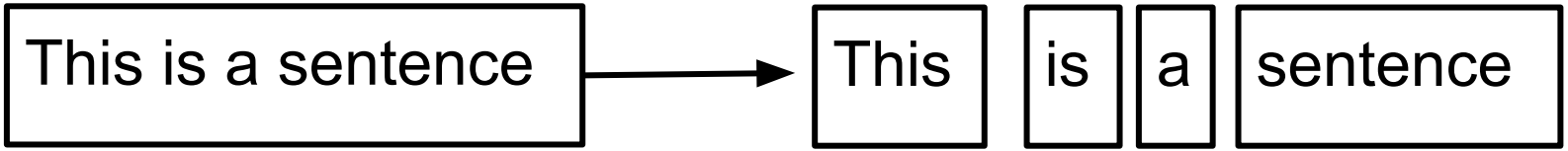
Word2Vec, Glove, ...

Text classification in general



How to work with textual data?

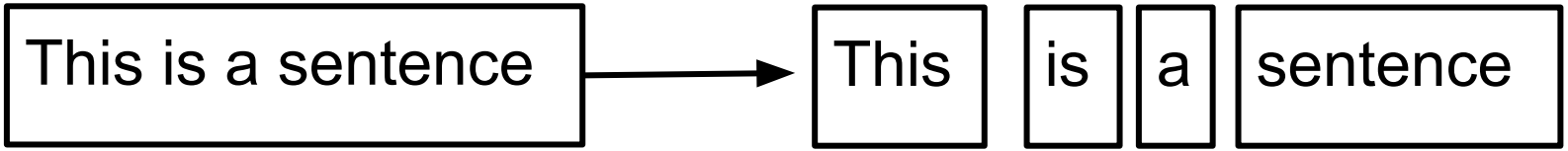
- What is textual data?



- Text == sequence of elements (tokens / embeddings / etc.)
- Why standard Neural Networks are not a good choice?

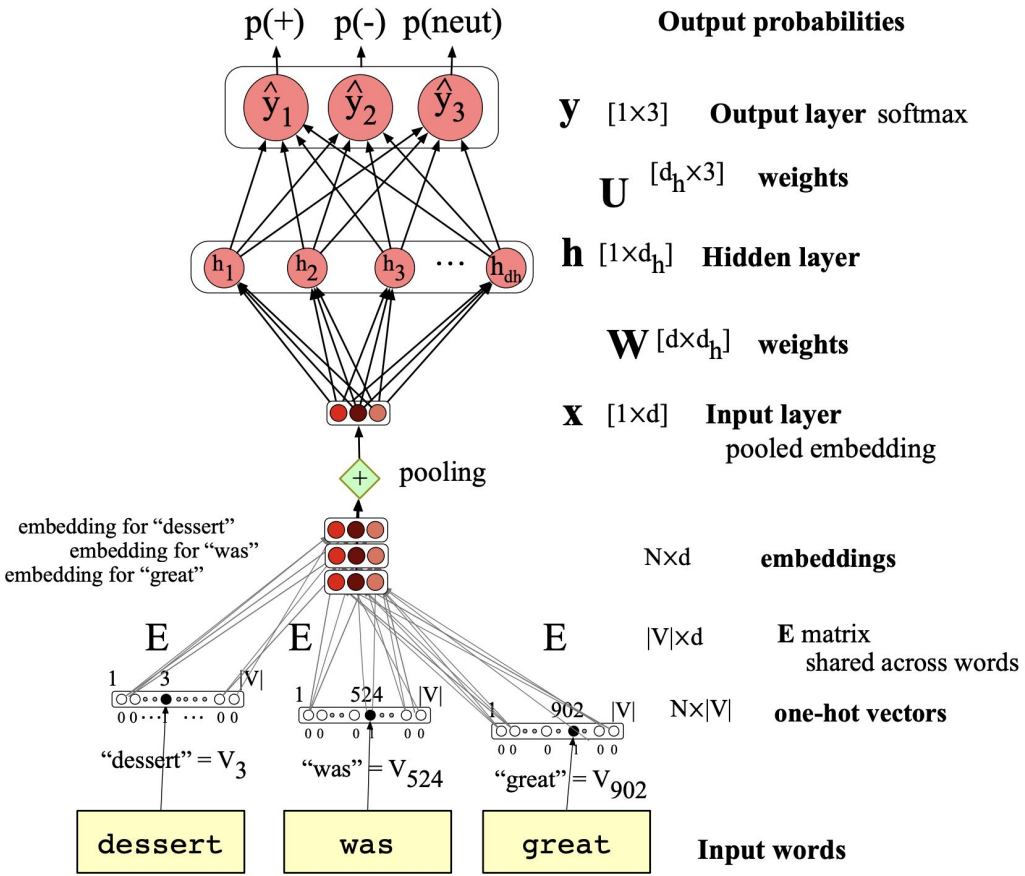
How to work with textual data?

- What is textual data?

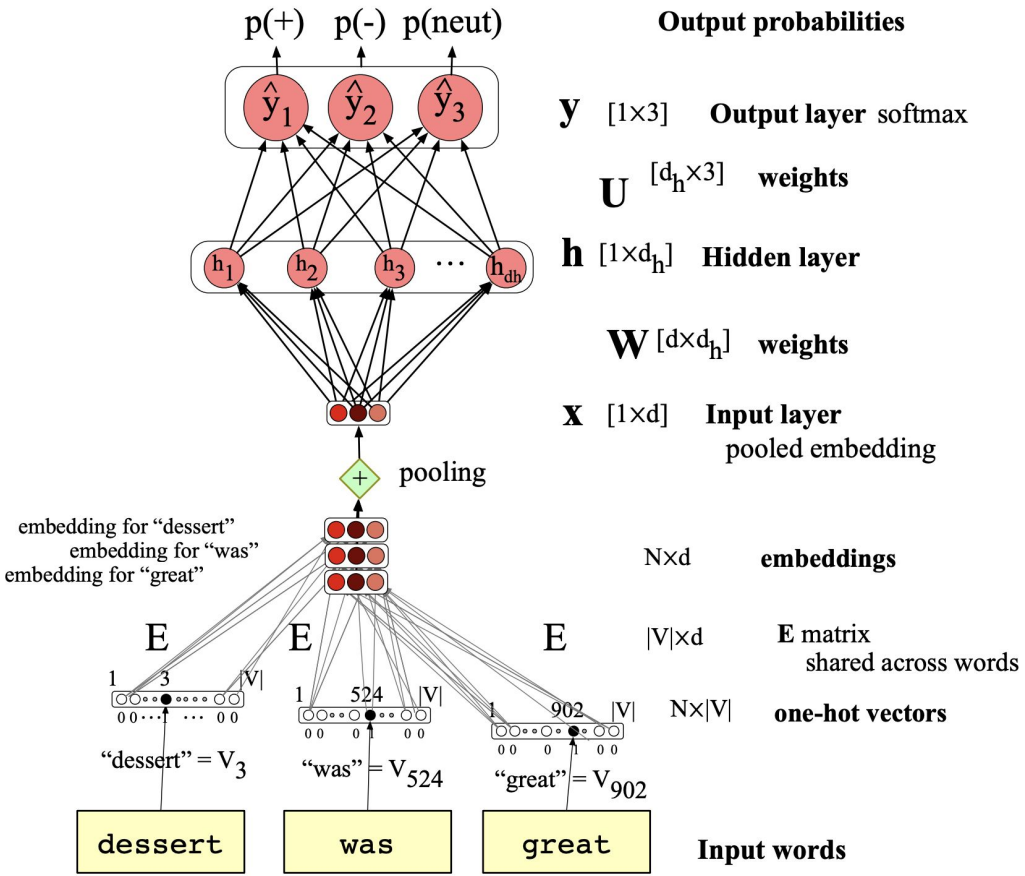


- Text == sequence of elements (tokens / embeddings / etc.)
- Why standard Neural Networks are not a good choice?

How to work with textual data?



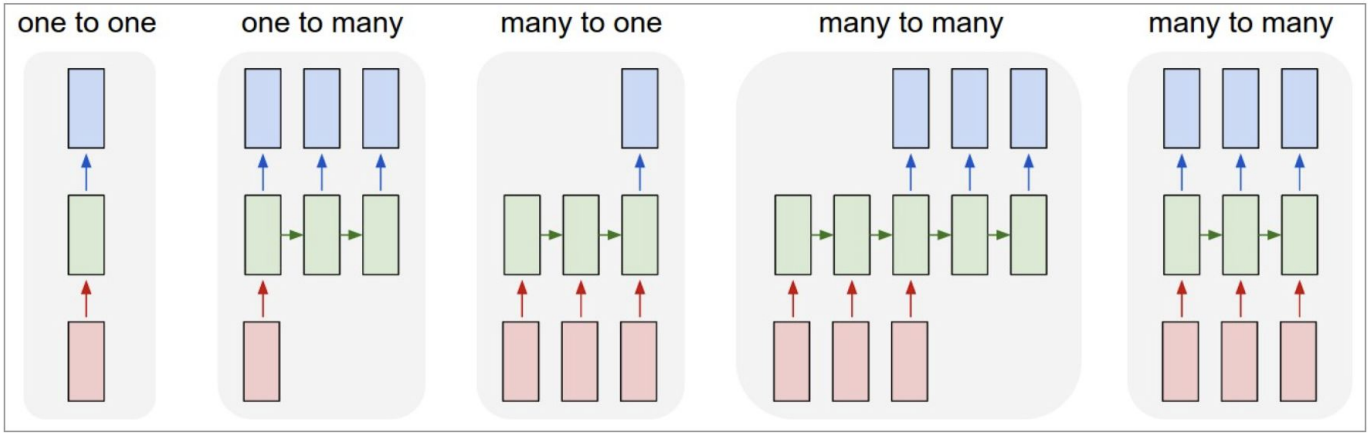
How to work with textual data?



- Order is not relevant
- Tokens do not influence each other
- "Bag of tokens"

Reccurrent Neural Network (RNN)

- Reccurrent Neural Network (RNN) for sequence learning

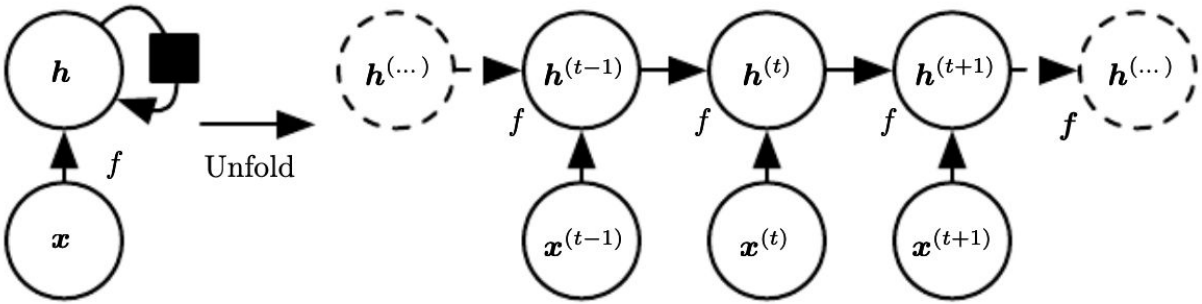


Reccurrent Neural Network (RNN)

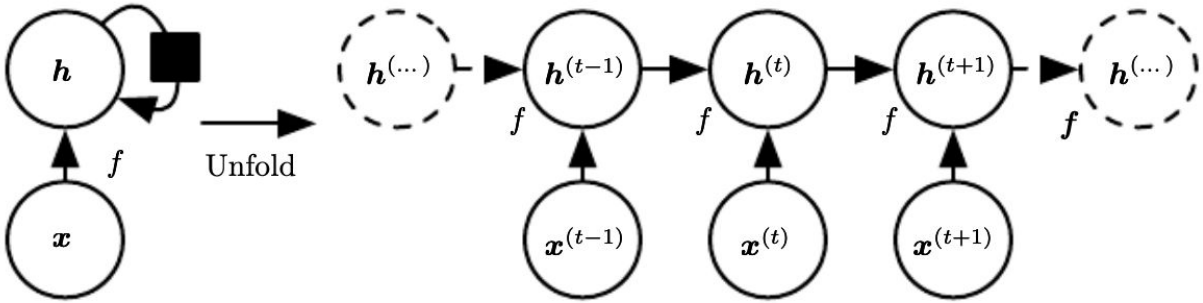
- Reccurrent Neural Network (RNN) for sequence learning
  - RNN processes sequential data one token at a time
  - A global “state” encodes all the meaning of the sequence
  - State is updated with each new token

Reccurrent Neural Network (RNN)

- Reccurrent Neural Network (RNN) for sequence learning
  - RNN processes sequential data one token at a time
  - A global “state” encodes all the meaning of the sequence
  - State is updated with each new token



Reccurrent Neural Network (RNN)



$$\boldsymbol{s}^{(t)} = f(\boldsymbol{s}^{(t-1)}, \boldsymbol{x}^{(t)}; \boldsymbol{\theta}),$$

$$\boldsymbol{h}^{(t)} = f(\boldsymbol{h}^{(t-1)}, \boldsymbol{x}^{(t)}; \boldsymbol{\theta}),$$

$$\begin{aligned}\boldsymbol{h}^{(t)} &= g^{(t)}(\boldsymbol{x}^{(t)}, \boldsymbol{x}^{(t-1)}, \boldsymbol{x}^{(t-2)}, \ldots, \boldsymbol{x}^{(2)}, \boldsymbol{x}^{(1)}) \\ &= f(\boldsymbol{h}^{(t-1)}, \boldsymbol{x}^{(t)}; \boldsymbol{\theta}).\end{aligned}$$

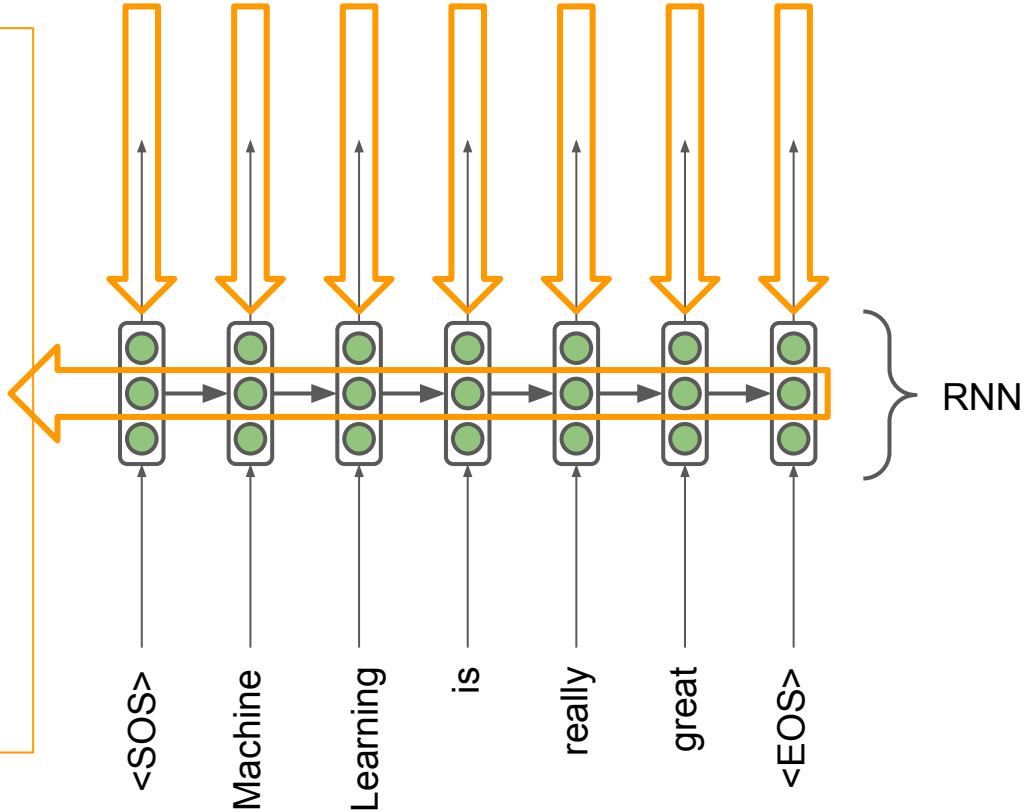
Activation functions

- ReLU - not okay because of matrix powers
- sigmod/tangent - ok because they are normed

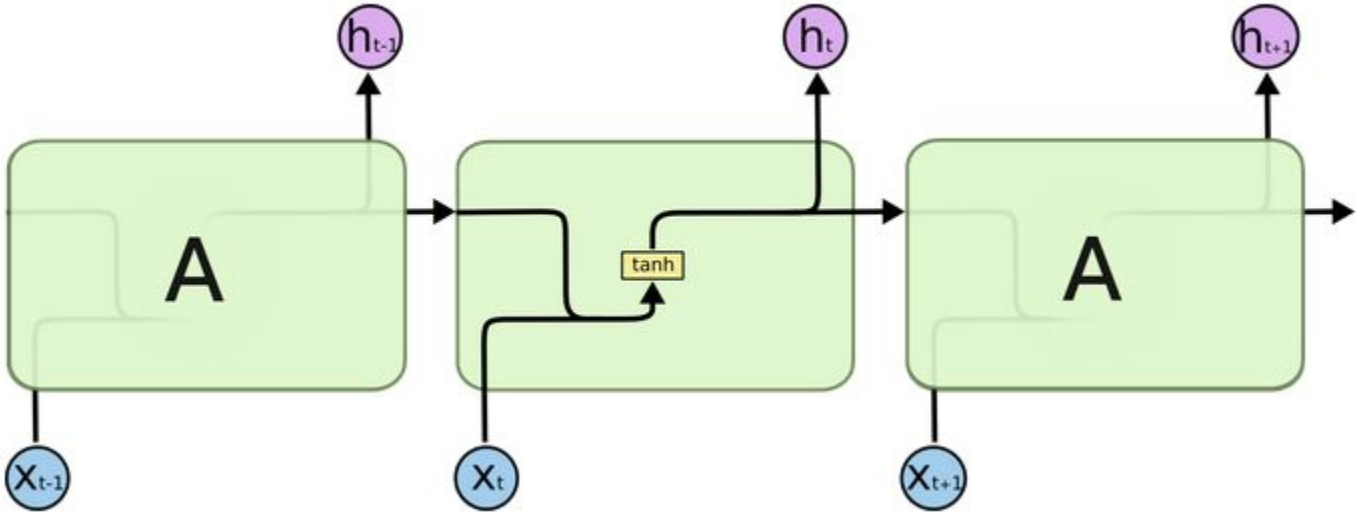
How to train it?

Loss

(e.g. Negative log-likelihood)



Vanilla RNN



RNN Pytorch implementation

Main hyperparameters:

- **input_size** – The number of expected features in the input x
- **hidden_size** – The number of features in the hidden state h
- **num_layers** – Number of recurrent layers. Default: 1
- **dropout** – If non-zero, introduces a Dropout layer on the outputs of each RNN layer except the last layer, with dropout probability equal to dropout. Default: 0
- **bidirectional** – If True, becomes a bidirectional RNN. Default: False

https://pytorch.org/docs/stable/generated/torch.nn.RNN.html

$$h_t = \tanh(x_t W_{ih}^T + b_{ih} + h_{t-1} W_{hh}^T + b_{hh})$$

Exploding gradient problem

If the gradient becomes too big, then the SGD update step becomes too big:

$$\theta^{new} = \theta^{old} - \overbrace{\alpha}^{\text{learning rate}} \underbrace{\nabla_\theta J(\theta)}_{\text{gradient}}$$

This can cause bad updates: we take too large a step and reach a bad parameter configuration (with large loss).

In the worst case, this will result in Inf or NaN in your network (then you have to restart training from an earlier checkpoint).

Exploding gradient solution

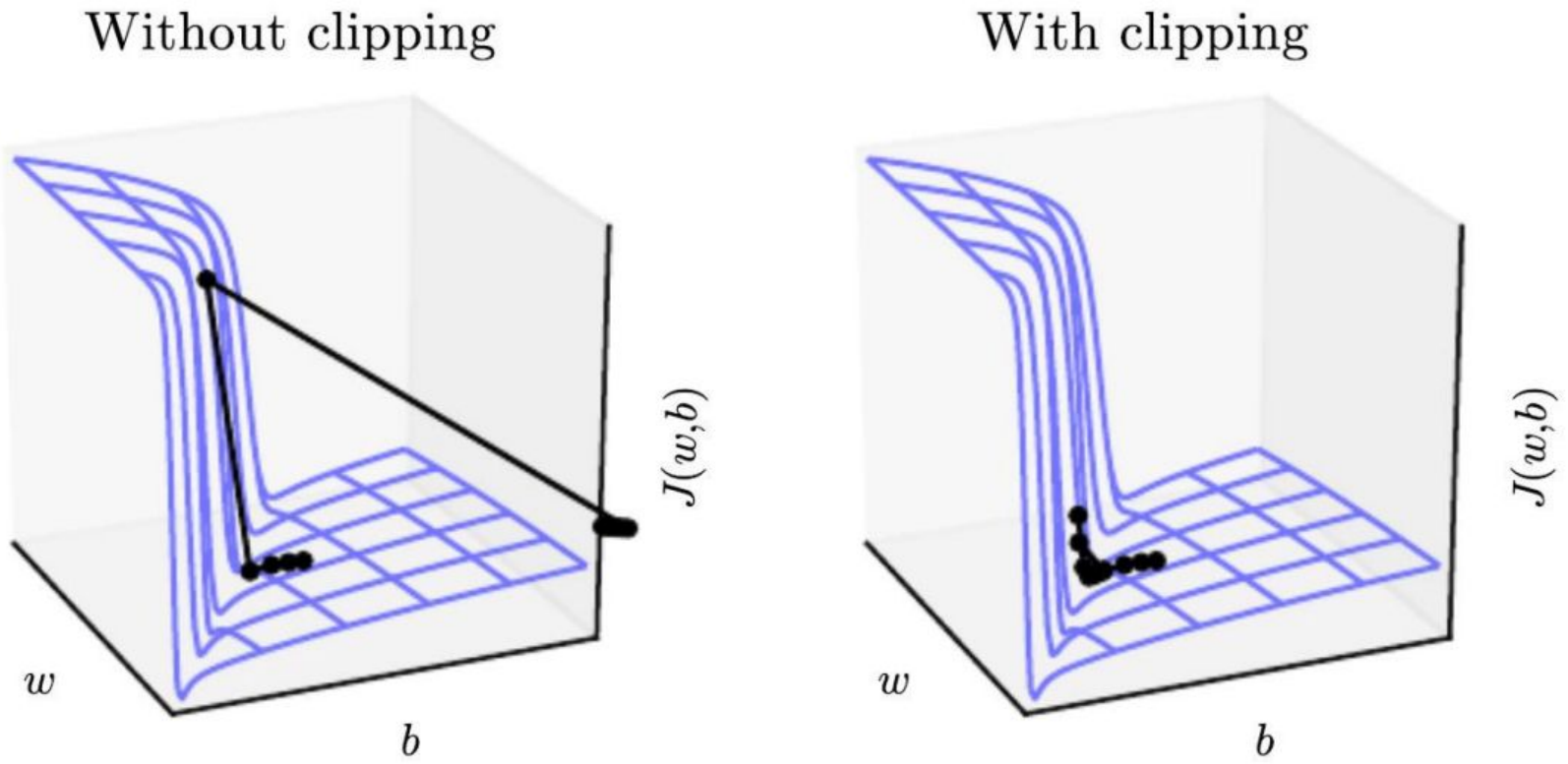
- Gradient clipping: if the norm of the gradient is greater than some threshold, scale it down before applying SGD update

Algorithm 1 Pseudo-code for norm clipping

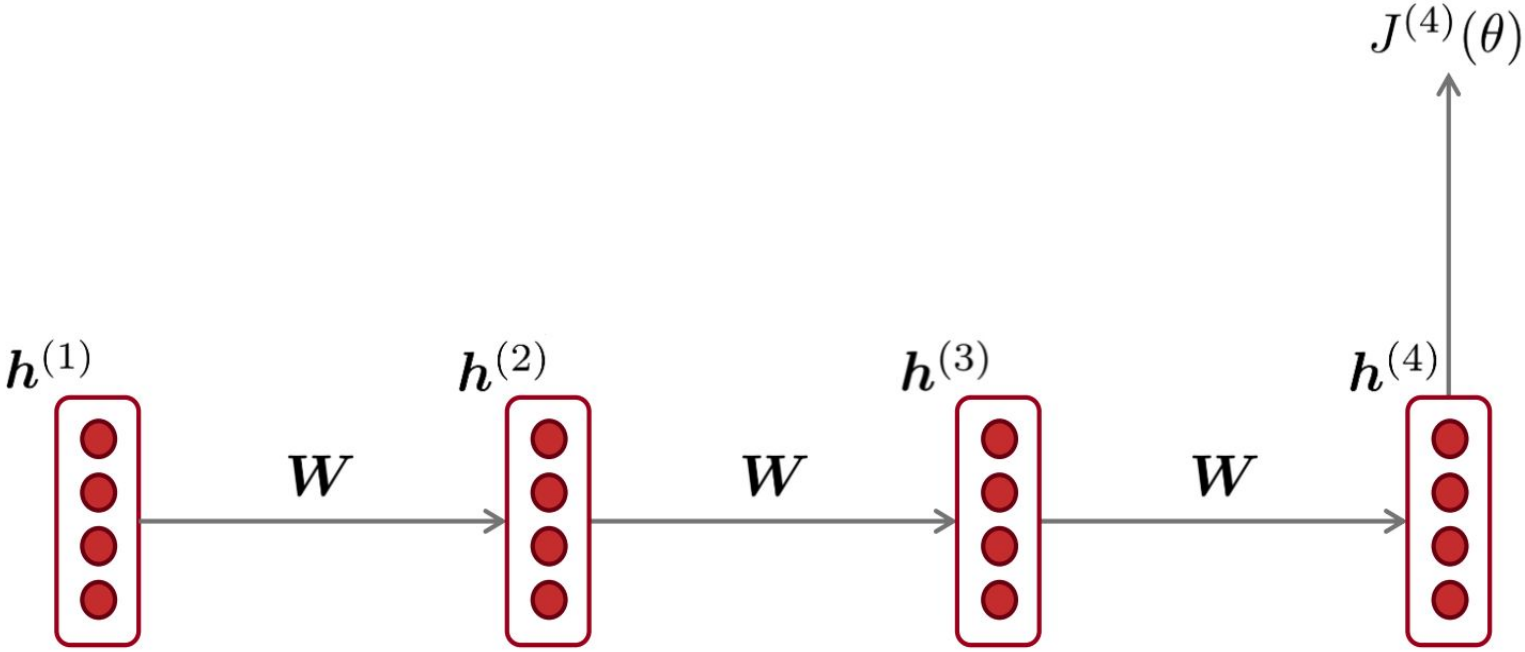
$\hat{\mathbf{g}} \leftarrow \frac{\partial \mathcal{E}}{\partial \theta}$
if $\|\hat{\mathbf{g}}\| \geq threshold$ **then**
  $\hat{\mathbf{g}} \leftarrow \frac{threshold}{\|\hat{\mathbf{g}}\|}\hat{\mathbf{g}}$
end if

- Intuition: take a step in the same direction, but a smaller step

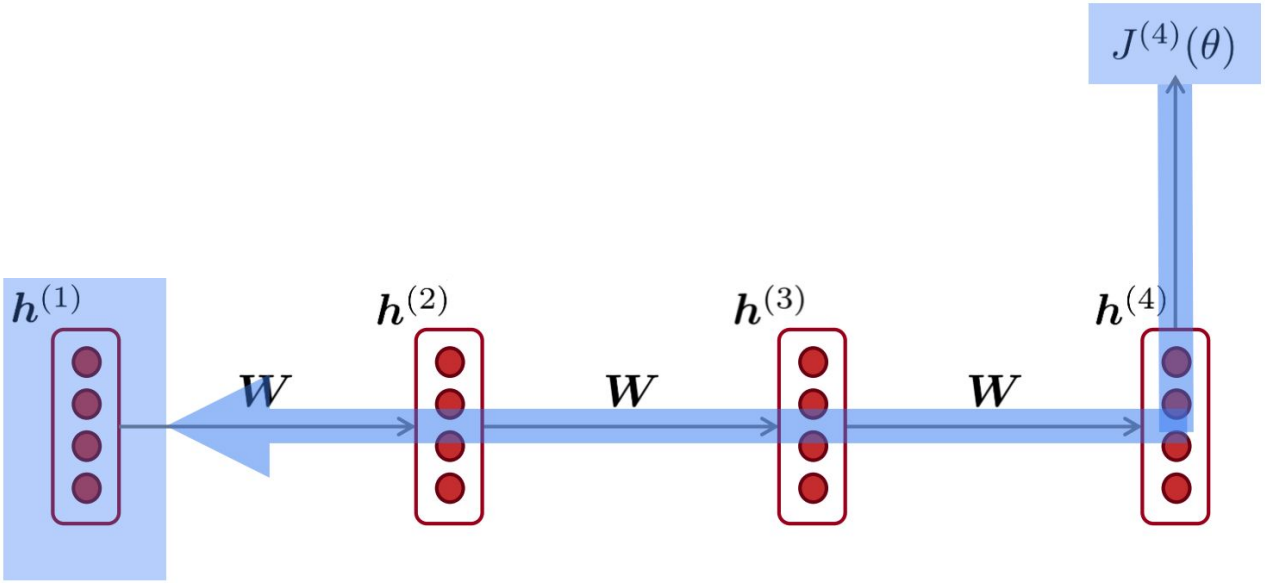
Exploding gradient solution



Vanishing gradient problem

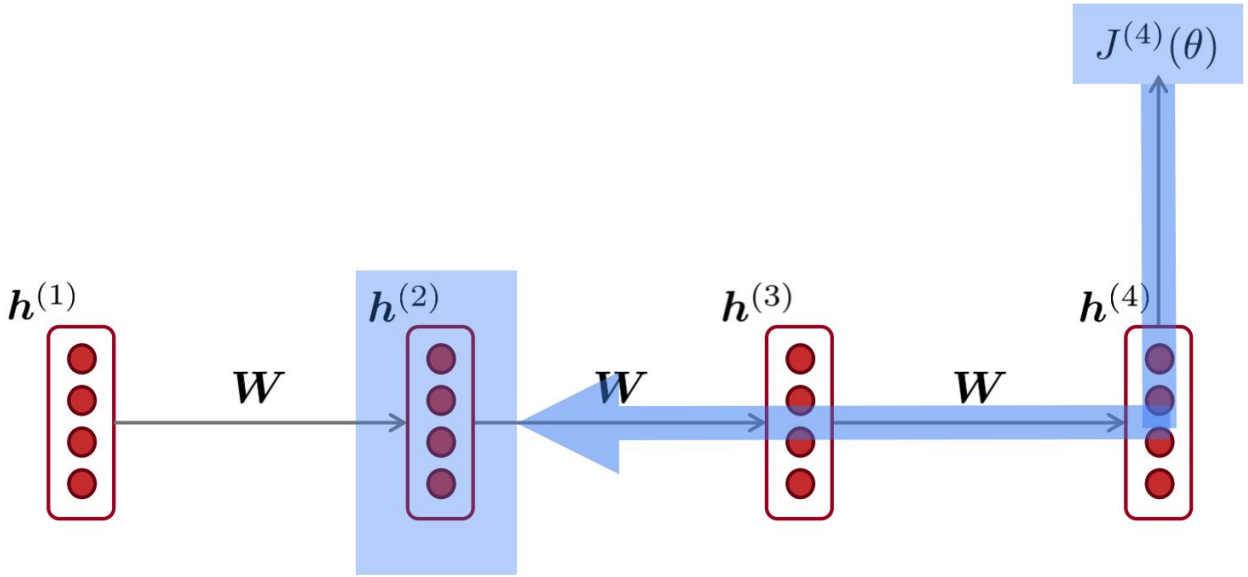


Vanishing gradient problem



$$\frac{\partial J^{(4)}}{\partial h^{(1)}} = \text{?}$$

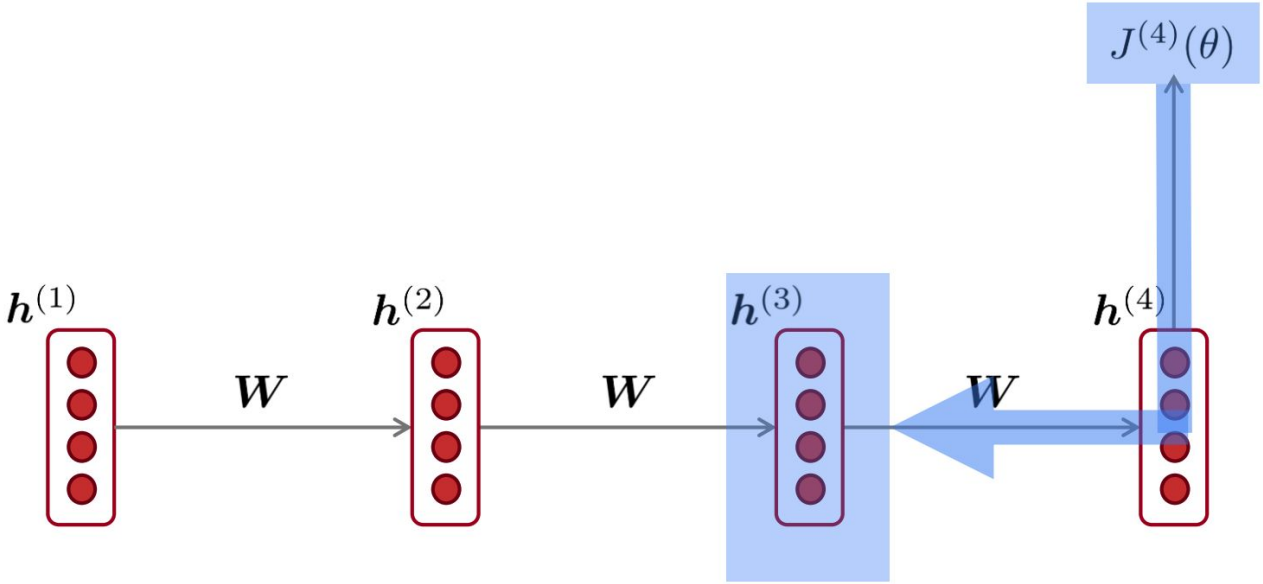
Vanishing gradient problem



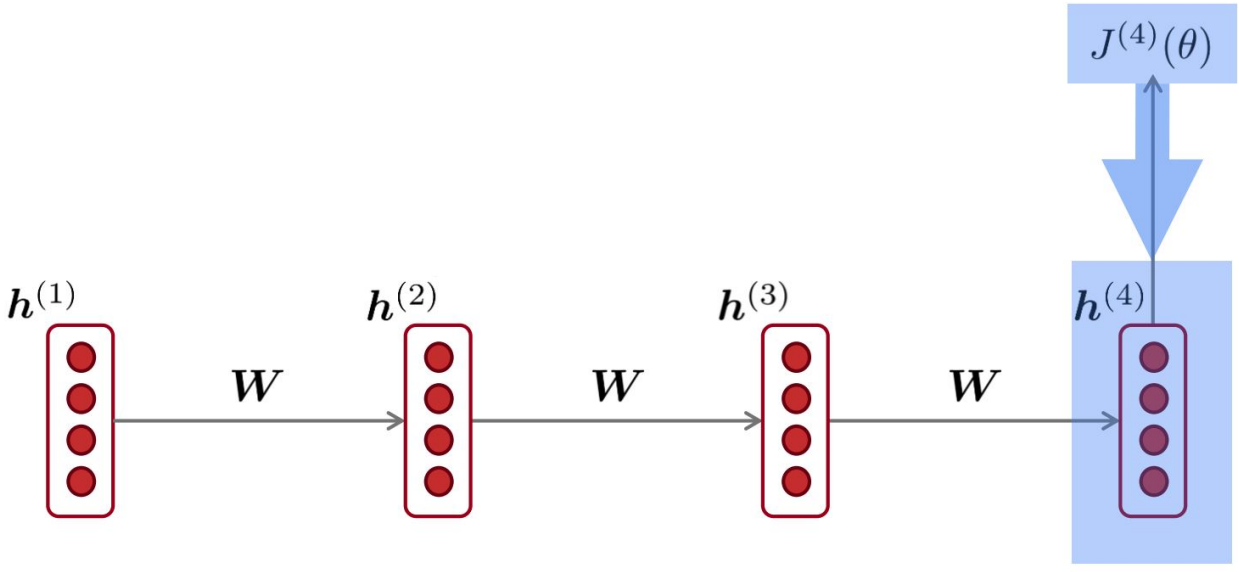
$$\frac{\partial J^{(4)}}{\partial h^{(1)}} = \frac{\partial h^{(2)}}{\partial h^{(1)}} \times \frac{\partial J^{(4)}}{\partial h^{(2)}}$$

chain rule!

Vanishing gradient problem



Vanishing gradient problem



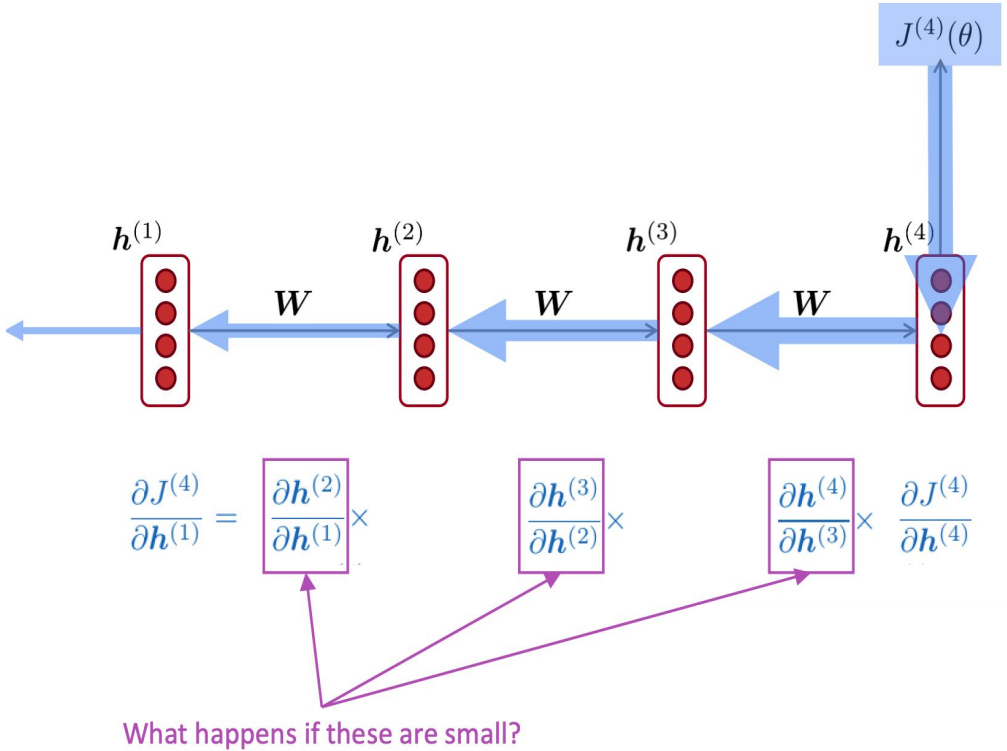
$$\frac{\partial J^{(4)}}{\partial h^{(1)}} = \frac{\partial h^{(2)}}{\partial h^{(1)}} \times \frac{\partial h^{(3)}}{\partial h^{(2)}} \times \frac{\partial h^{(4)}}{\partial h^{(3)}} \times \frac{\partial J^{(4)}}{\partial h^{(4)}}$$

chain rule!

Vanishing gradient problem

Vanishing gradient problem:

When the derivatives are small, the gradient signal gets smaller and smaller as it backpropagates further

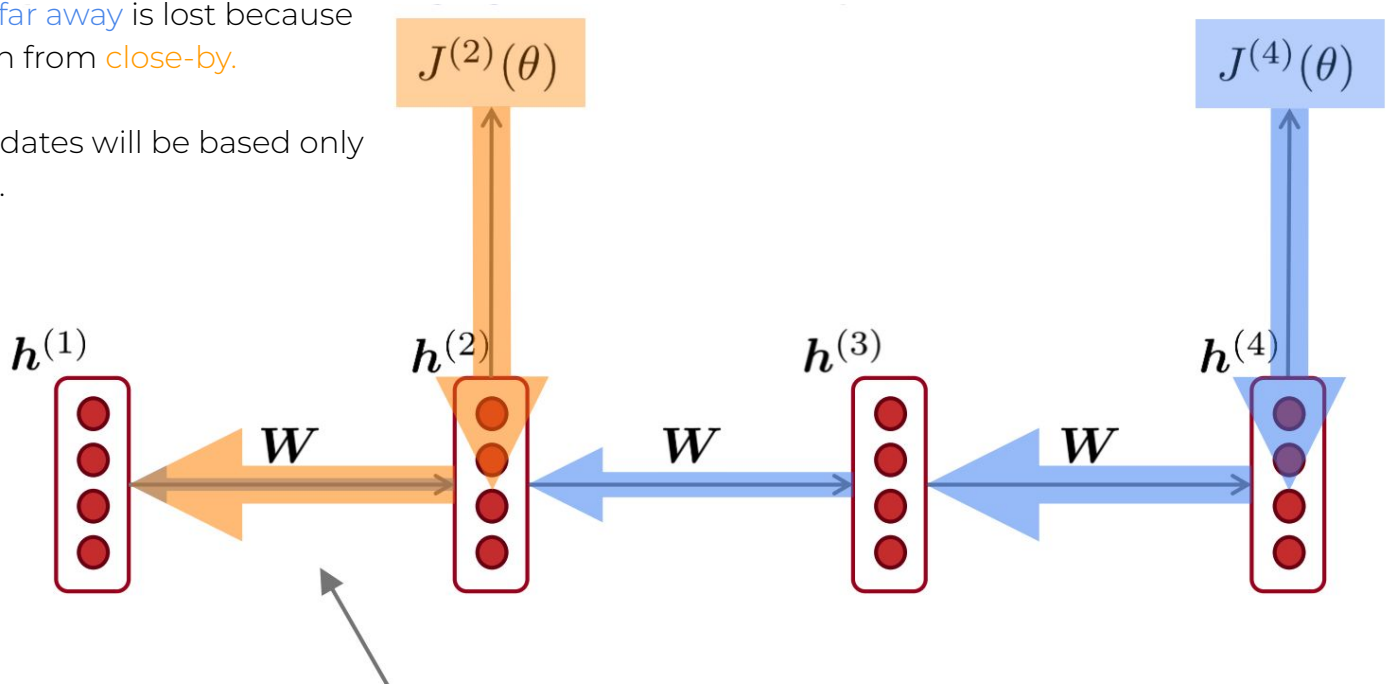


More info: "On the difficulty of training recurrent neural networks", Pascanu et al, 2013
http://proceedings.mlr.press/v28/pascanu13.pdf

Vanishing gradient problem

Gradient signal from far away is lost because it's much smaller than from close-by.

So model weights updates will be based only on short-term effects.



Vanishing gradient in non-RNN

Vanishing gradient is present in all deep neural network architectures.

- Due to chain rule / choice of nonlinearity function, gradient can become vanishingly small during backpropagation
- Lower levels are hard to train and are trained slower
- **Potential solution(but not actually for that problem):** dense connections (just like in DenseNet)

Conclusion:

Though vanishing/exploding gradients are a general problem, RNNs are particularly unstable due to the repeated multiplication by the same weight matrix [Bengio et al, 1994]. Gradients magnitude drops exponentially with connection length.

Vanishing gradient in non-RNN

Vanishing gradient is present in **all** deep neural network architectures.

- Due to chain rule / choice of nonlinearity function, gradient can become vanishingly small during backpropagation
- Lower levels are hard to train and are trained slower
- **Potential solution:** direct (or skip-) connections (just like in ResNet)

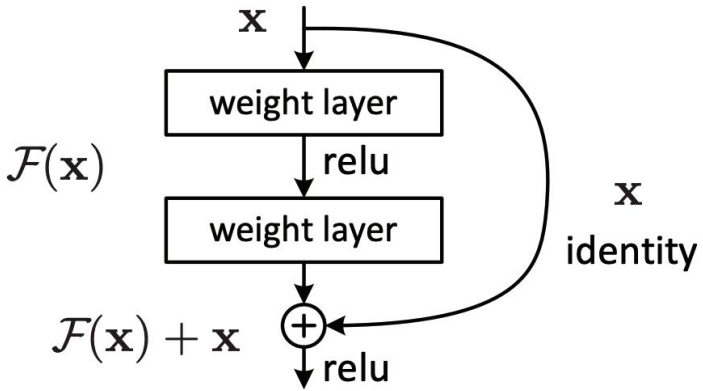


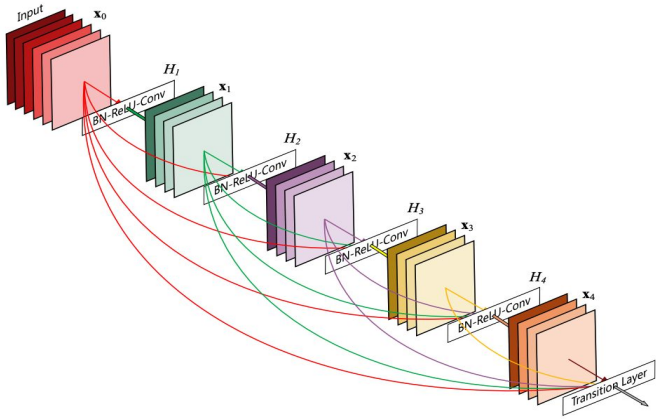
Figure 2. Residual learning: a building block.

Source: "Deep Residual Learning for Image Recognition", He et al, 2015.
https://arxiv.org/pdf/1512.03385.pdf

Vanishing gradient in non-RNN

Vanishing gradient is present in **all** deep neural network architectures.

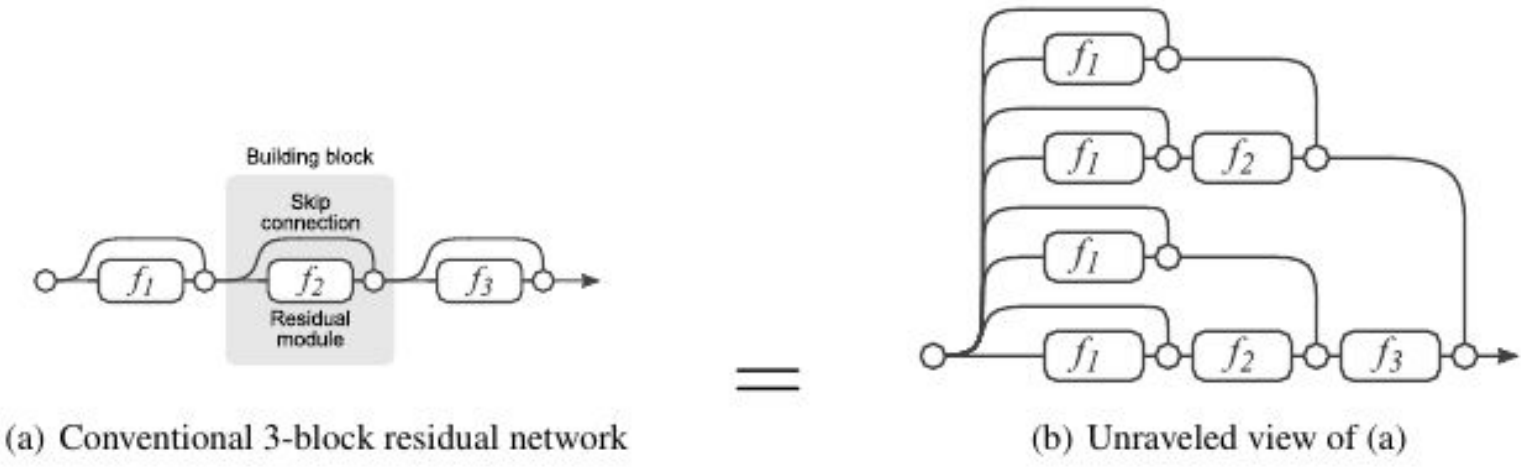
- Due to chain rule / choice of nonlinearity function, gradient can become vanishingly small during backpropagation
- Lower levels are hard to train and are trained slower
- **Potential solution:** dense connections (just like in DenseNet)



Source: "Densely Connected Convolutional Networks", Huang et al, 2017
https://arxiv.org/pdf/1608.06993.pdf

Another view on ResNets and vanishing gradient

"Residual Networks Behave Like Ensembles of Relatively Shallow Networks"



(a) Conventional 3-block residual network

(b) Unraveled view of (a)

Source: https://arxiv.org/pdf/1605.06431.pdf

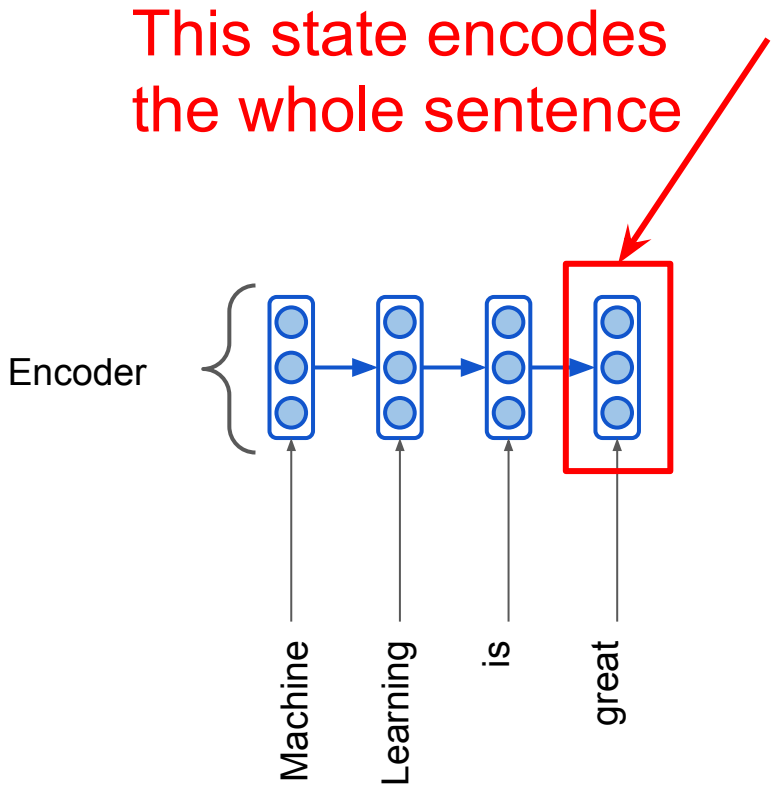
Neural Machine Translation

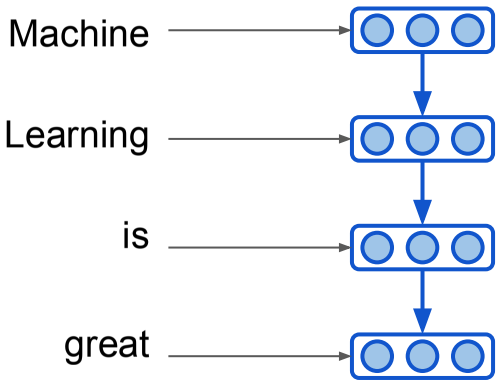
What is Neural Machine Translation?

- Neural Machine Translation (NMT) is a way to do Machine Translation with a single neural network

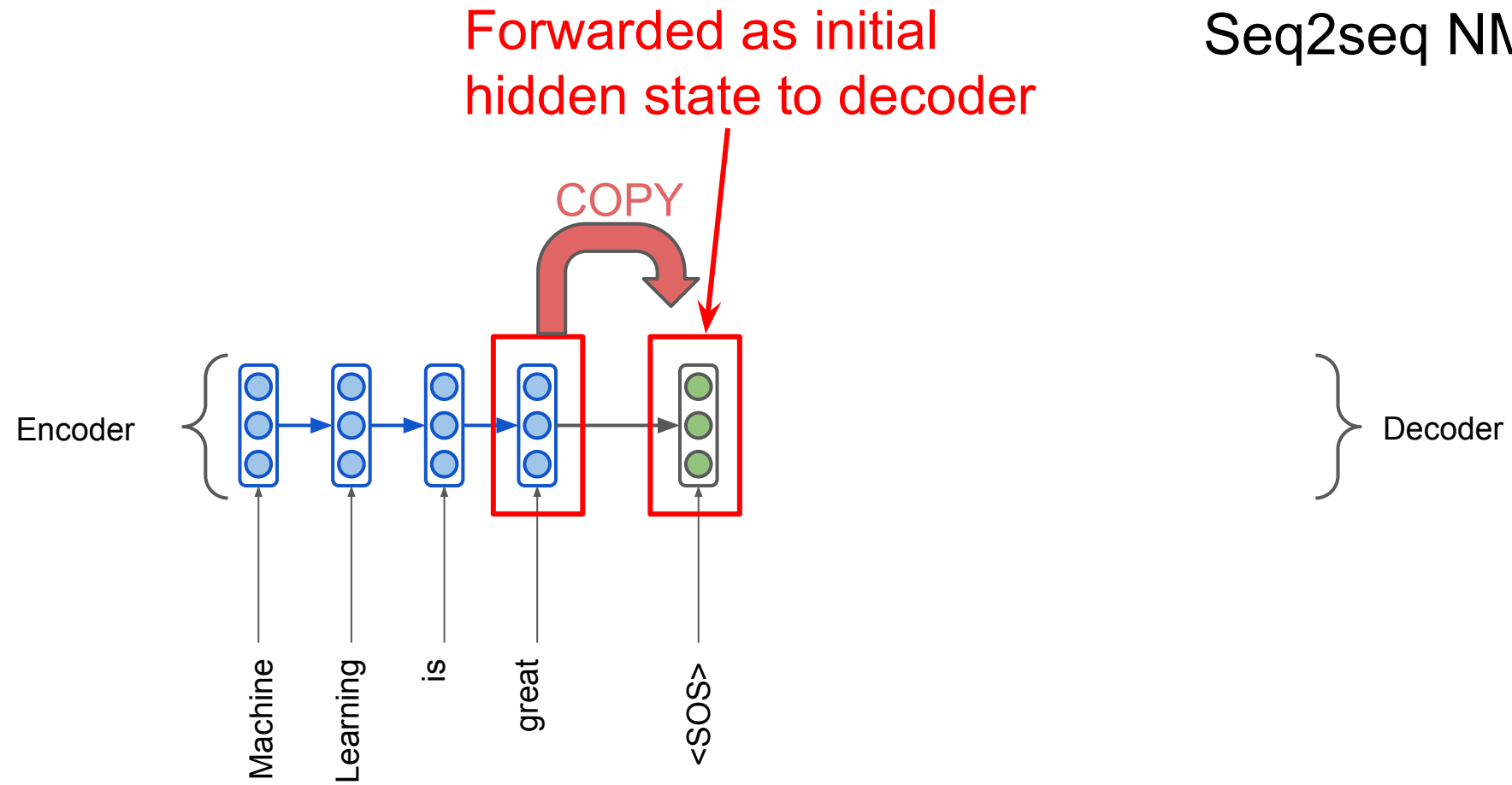
- The neural network architecture is called sequence-to-sequence (aka **seq2seq**), it involves two **RNNs**

Seq2seq NMT

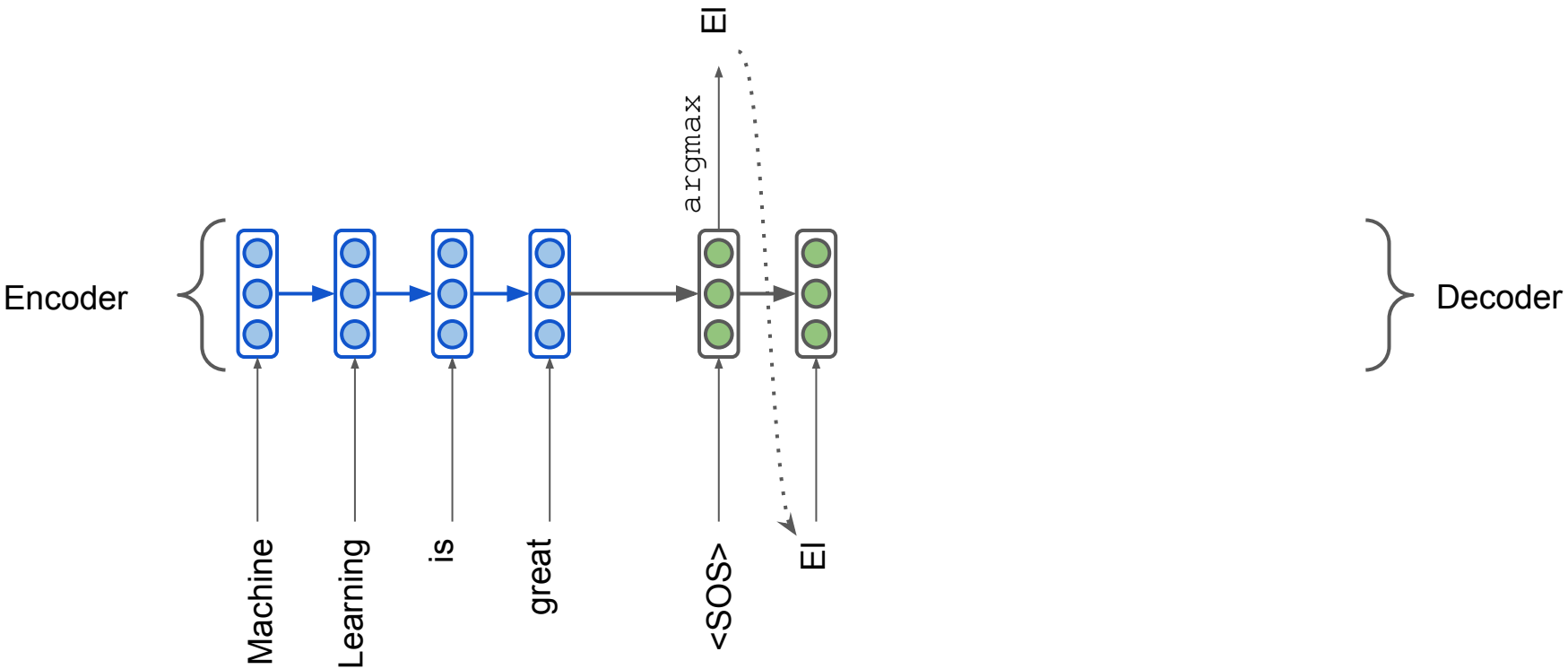

This state encodes
the whole sentence
Encoder
Machine
Learning
is
great


Machine
Learning
is
great

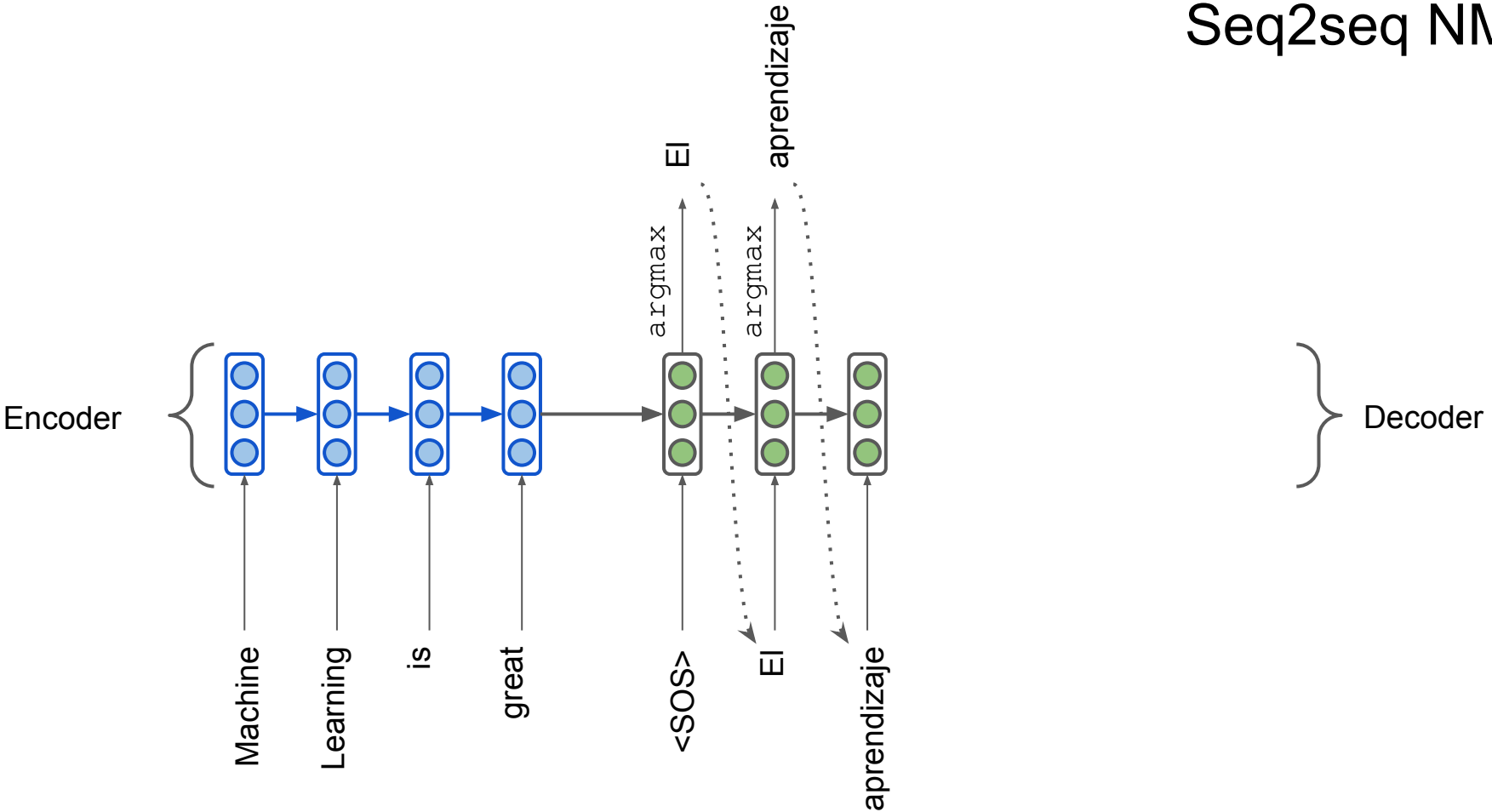
Seq2seq NMT


Forwarded as initial
hidden state to decoder
COPY
Encoder
Decoder
Machine
Learning
is
great
<SOS>

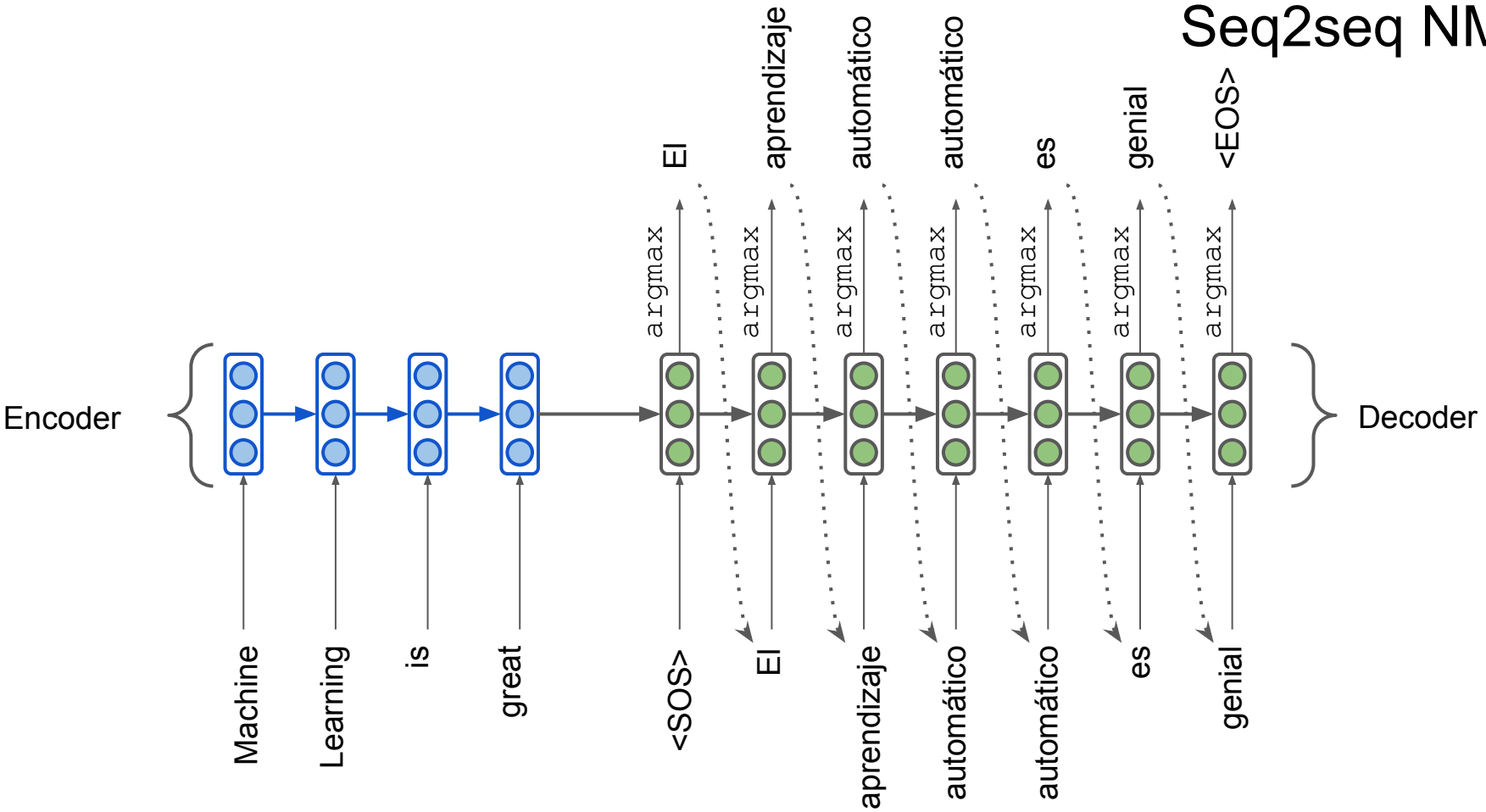
Seq2seq NMT



Seq2seq NMT


Encoder
Machine
Learning
is
great
<SOS>
argmax
El
El
argmax
aprendizaje
aprendizaje
Decoder

Seq2seq NMT


Encoder
Machine
Learning
is
great
<SOS>
El
aprendizaje
automático
automático
es
genial
<EOS>
argmax
Decoder

NMT: how does it work?

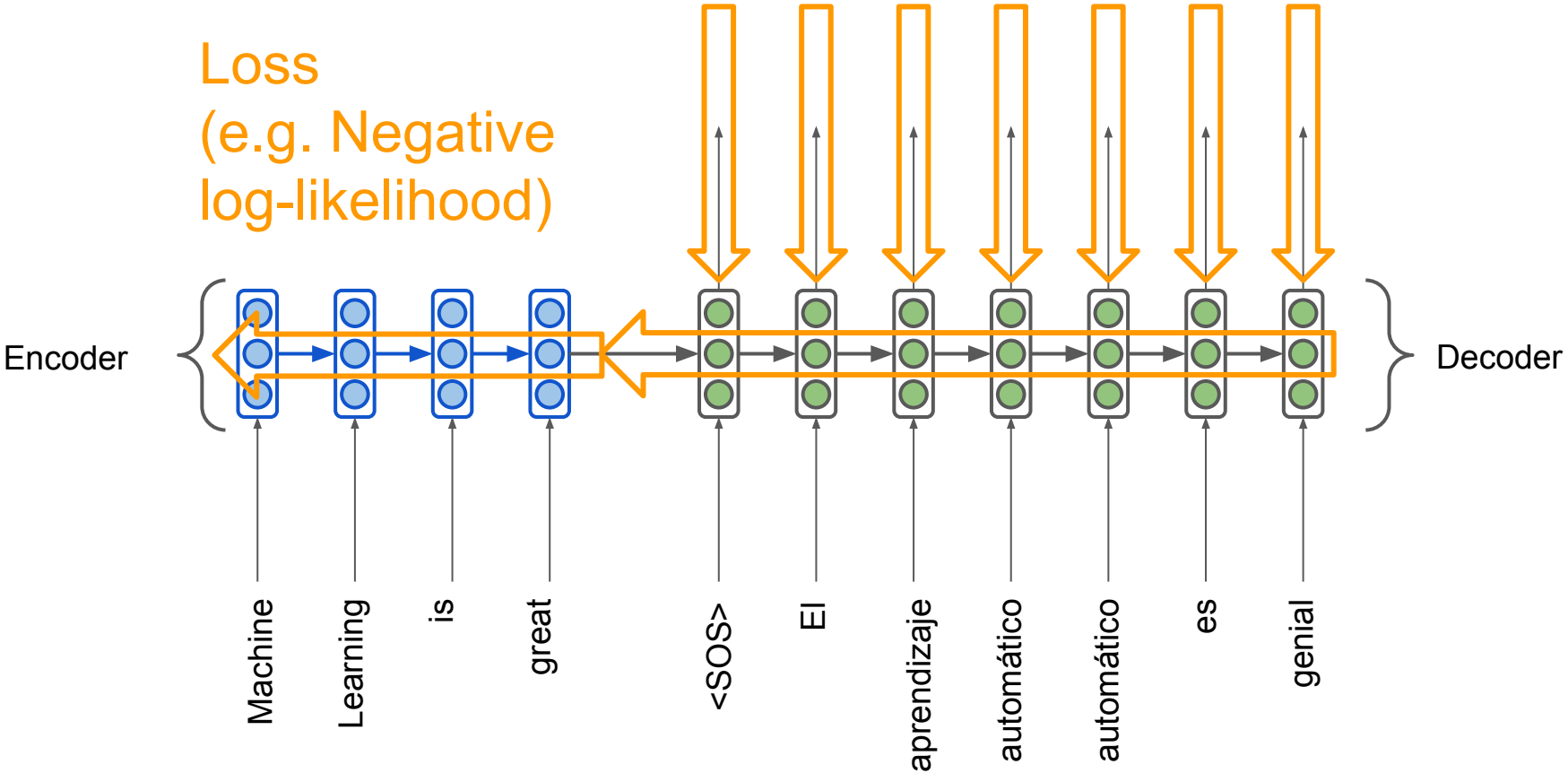
- NMT directly calculates P(y|x)
  - y – target sentence, x – source sentence

$$P(y|x) = P(y_2|y_1, x)P(y_3|y_1, y_2, x)\ldots \underbrace{P(y_T|y_1, y_2, \ldots, x)}$$

Probability of next word
in target language

- To train it we need a huge parallel corpus.

Seq2seq is trained end-to-end

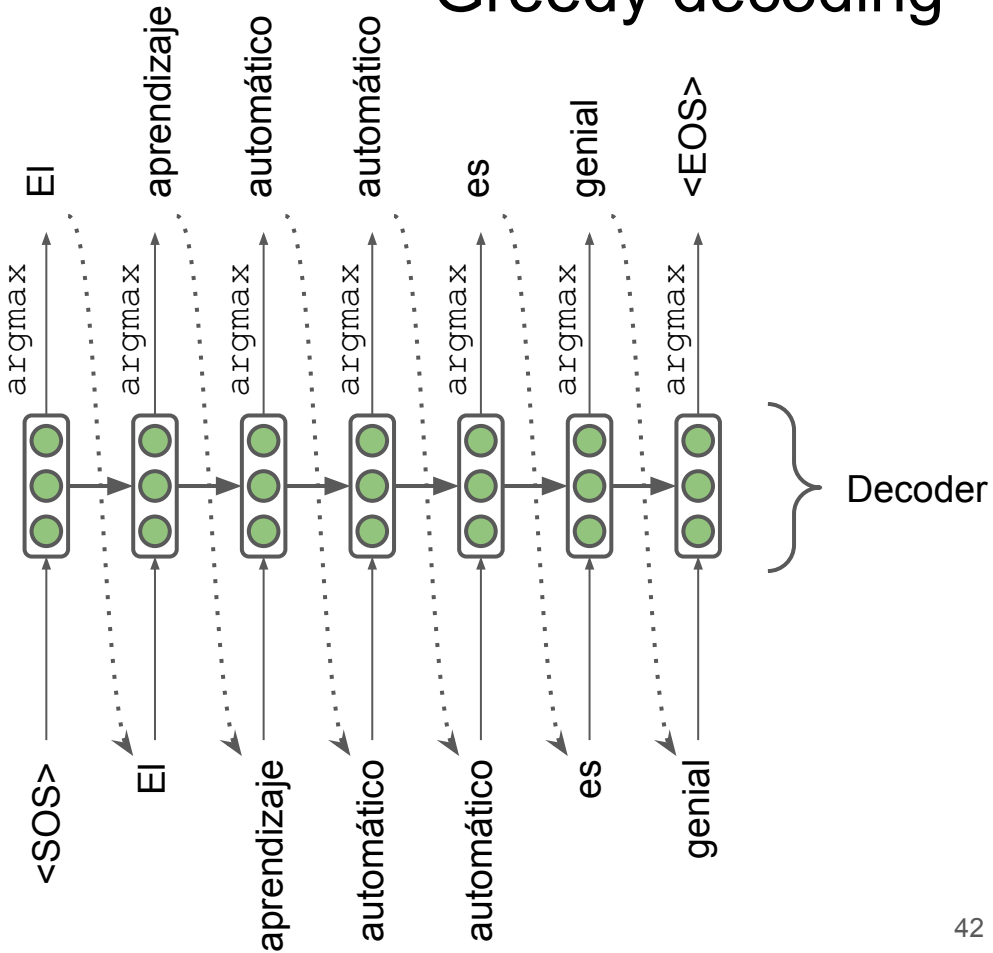


Greedy decoding

- Decoder predicts the most probable token (argmax) on each step
- The approach is **greedy**

Any problems with it?

Any mistake is treated as input on the next step!



Exhaustive search

- We want the translation that maximizes the likelihood:

$$P(y|x) = P(y_1|x) \prod_{t=2}^{T} P(y_t|y_1, \dots y_{t-1}, x)$$

- We cannot compute all the possible sequences (exponential complexity)

Beam search

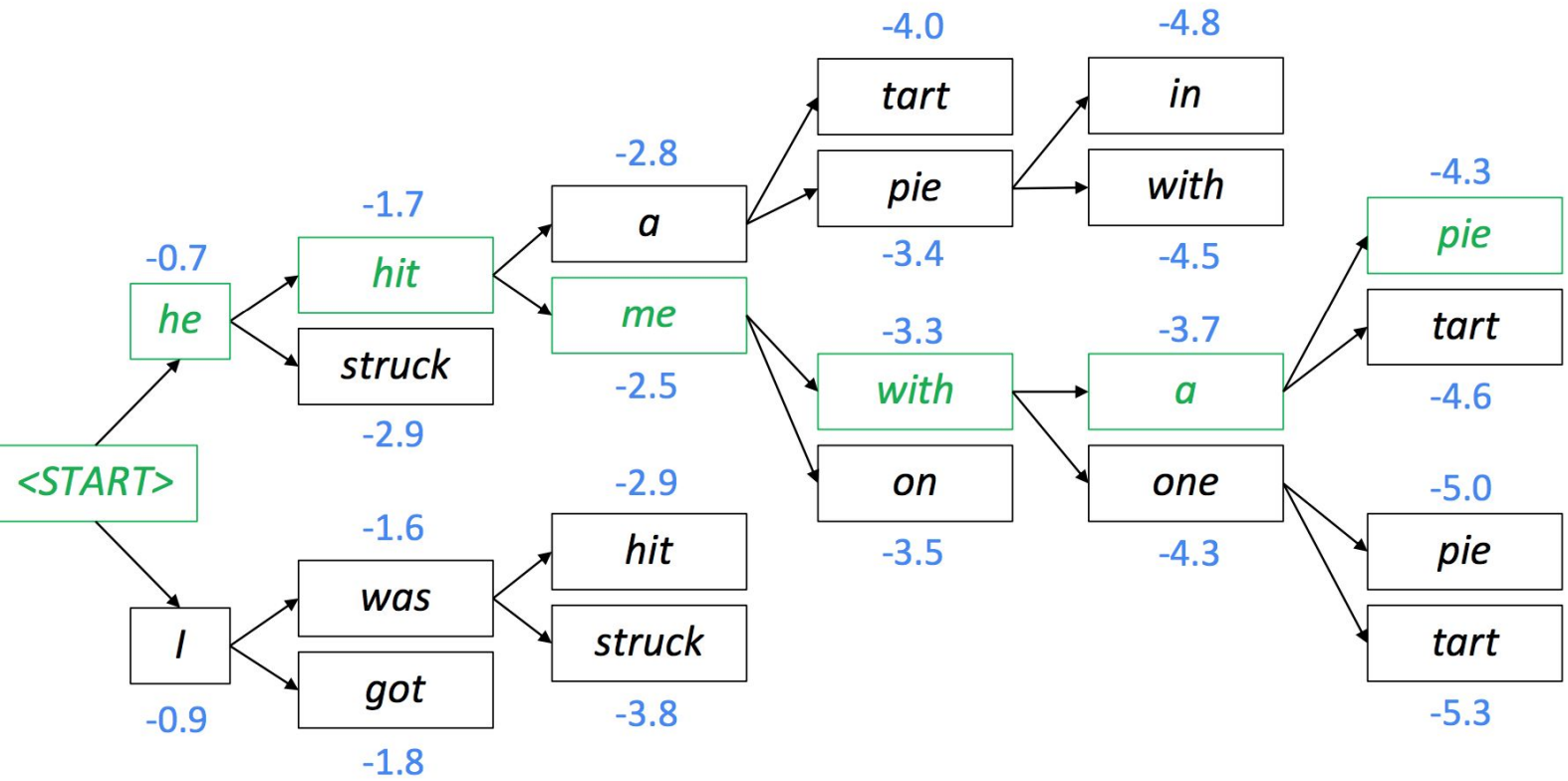
- On each step of decoder, keep track of the k most probable partial translations (which we call hypotheses)
- k is the beam size (in practice around 5 to 10)
- A hypothesis has a score which is its log probability:

$$\text{score}(y_1, \ldots, y_t) = \log P_{\text{LM}}(y_1, \ldots, y_t | x) = \sum_{i=1}^{t} \log P_{\text{LM}}(y_i | y_1, \ldots, y_{i-1}, x)$$

- We search for high-scoring hypotheses, tracking top k on each step
- Beam search does not guarantee finding optimal solution

Beam search decoding: example

Beam size = k = 2. Blue numbers = $\text{score}(y_1, \ldots, y_t) = \sum_{i=1}^{t} \log P_{\text{LM}}(y_i | y_1, \ldots, y_{i-1}, x)$



Source: http://web.stanford.edu/class/cs224n/slides/cs224n-2019-lecture08-nmt.pdf

Beam search decoding: stopping criterion

- In **greedy decoding**, usually we decode until the model produces <EOS> token
- In **beam search decoding**, different hypotheses may produce <EOS> tokens on different timesteps
  - When a hypothesis produces <EOS>, that hypothesis is complete.
  - Place it aside and continue exploring other hypotheses via beam search.
- Usually we continue beam search until:
  - We reach pre-defined timestep T
  - We have at least n completed hypotheses

Beam search decoding: finishing up

- How to select top one with highest score?

$$\text{score}(y_1, \ldots, y_t) = \log P_{\text{LM}}(y_1, \ldots, y_t | x) = \sum_{i=1}^{t} \log P_{\text{LM}}(y_i | y_1, \ldots, y_{i-1}, x)$$

- **Problems?**

  Longer hypotheses have lower scores!

$$\frac{1}{t} \sum_{i=1}^{t} \log P_{\text{LM}}(y_i | y_1, \ldots, y_{i-1}, x)$$

NMT: Quality Evaluation

BLEU

BLEU (Bilingual Evaluation Understudy) compares the machine-written translation to human-written translation, and computes a similarity score based on:

- n-gram precision
- penalty for too-short system translations (brevity penalty)

$$BLEU = \text{brevity penalty} \cdot \left(\prod_{i=1}^{n} \text{precision}_i \right)^{1/n} \cdot 100\%$$

$$\text{brevity penalty} = min \left(1, \frac{\text{output length}}{\text{reference length}} \right)$$

BLEU

BLEU (Bilingual Evaluation Understudy) compares the machine-written translation to human-written translation, and computes a similarity score based on:

- n-gram precision
- brevity penalty

SYSTEM A: Israeli officials (2-GRAM MATCH) responsibility of airport (1-GRAM MATCH) safety

REFERENCE: Israeli officials are responsible for airport security

SYSTEM B: airport security (2-GRAM MATCH) Israeli officials are responsible (4-GRAM MATCH)

Metric	System A	System B
precision (1gram)	3/6	6/6
precision (2gram)	1/5	4/5
precision (3gram)	0/4	2/4
precision (4gram)	0/3	1/3
brevity penalty	6/7	6/7
BLEU	0%	52%

$$BLEU = \text{brevity penalty} \cdot \left(\prod_{i=1}^{n} \text{precision}_i \right)^{1/n} \cdot 100\%$$

BLEU

BLEU is imperfect:

- There are many valid ways to translate a sentence
- So a good translation may get a poor BLEU score just because of low n-gram overlap with the human translation

Quality metrics for MT

- **ROUGE** (Recall-Oriented Understudy for Gisting Evaluation)
- **METEOR** (Metric for Evaluation of Translation with Explicit ORdering)
  - Uses synonyms from WordNet

Quality metrics for MT

- **Human evaluation**

Side-by-side (SbS HE)

Q: Pick the best translaiton

Src: Beam search decoding
System1: декодирование с поиском лучадекодирование поиска луча
System2: декодирование поиска луча

Direct Assessment (DA)

Q: Rate this translation of 5-point scale
Src: Beam search decoding
Translation: декодирование поиска луча

Quality metrics for MT

Human eval is expensive

Solution: ML models trained to approximate HE

1. BLEURT (Google) - BERT finetuned on DA ratings
2. XCOMET (Unbabel) - XLM finetuned on DA + MQM ratings
3. LLM-based estimators - prompt LLM to estimate MT quality